

Tabla de contenido

Módulo 4 – Transformar.....2

 Clase 5. Diseño de assistants de AI adaptados a procesos reales.....2

 Resultados de aprendizaje observables.....2

 Parte 1.....2

 Chatbot, assistant, agent y automation.....2

 Arquitectura básica de assistants.....3

 Seguridad y protección de información.....4

 Parte 2.....5

 Instrucciones y proyectos de asistentes.....5

 Concepto de apoyo: Retrieval-Augmented Generation, embeddings y knowledge bases.....5

 Integración con GPT, Gemini y Copilot.....6

 Casos de uso pertinentes.....6

 Limitaciones, riesgos, controversias y errores frecuentes.....7

 Buenas prácticas.....7

 Demostración sugerida para el docente.....7

 Preguntas de reflexión o comprobación.....7

 Ideas clave para convertir en diapositivas.....7

 Fuentes utilizadas en esta clase.....7

Bibliografía.....8

Módulo 4 – Transformar

Clase 5. Diseño de assistants de AI adaptados a procesos reales

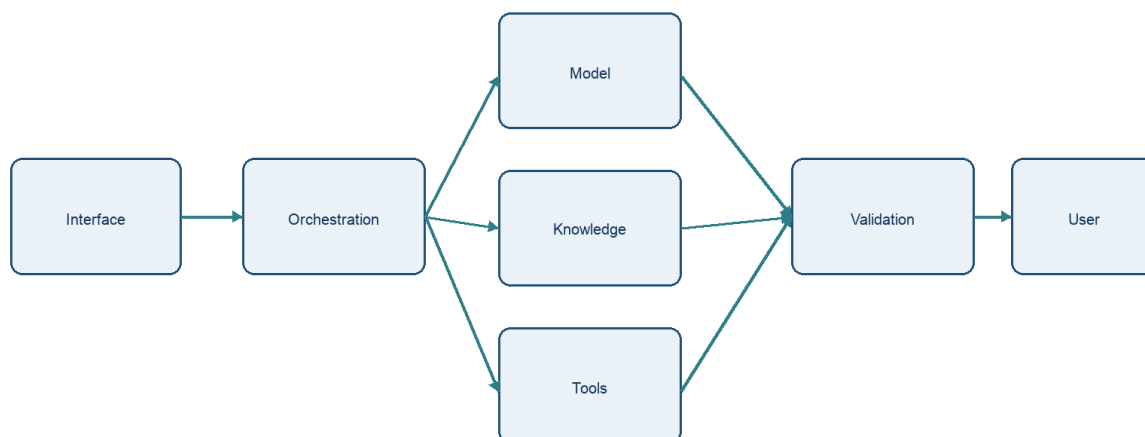
Propósito. Comprender la arquitectura, seguridad, instrucciones, conocimiento e integraciones necesarias para diseñar un asistente útil y gobernable.

Resultados de aprendizaje observables

- Distinguir chatbot, assistant, agent y automation por autonomía y capacidad de acción.
- Explicar una arquitectura básica con modelo, instrucciones, conocimiento, herramientas y controles.
- Diseñar principios de seguridad y protección de información.
- Explicar RAG y knowledge bases con sus límites.

Pregunta orientadora: ¿Qué convierte una conversación con un modelo en un asistente confiable para un proceso real?

Identidad, permisos, políticas, registros y supervisión rodean todo el sistema



*Figura 6. Arquitectura conceptual de un assistant. Elaboración propia con base en las fuentes citadas en la sección.
 Texto alternativo: Diagrama conceptual con bloques y flechas que resume la relación central de la clase.*

Parte 1

Chatbot, assistant, agent y automation

Explicación intuitiva. Un chatbot ofrece una interfaz conversacional. Un assistant combina conversación con instrucciones, contexto o knowledge sources para apoyar tareas. Un agent selecciona acciones o herramientas y mantiene estado para perseguir un objetivo con algún grado de autonomía. Una automation ejecuta un workflow definido cuando ocurre un trigger. Estas etiquetas no están estandarizadas y los proveedores las usan de forma diferente.

No forman una escala obligatoria. Un workflow determinista puede ser más seguro que un agent; un chatbot puede llamar herramientas; un agent puede activar automations. La pregunta útil es quién decide el siguiente paso, qué puede hacer, qué recuerda y quién aprueba.

Explicación técnica. Agency puede describirse mediante planning horizon, tool selection, state/memory, environmental feedback y authority. En un workflow, la ruta principal está codificada; en un agent, el modelo puede elegir dinámicamente la siguiente acción dentro de un conjunto. En la práctica existen diseños híbridos [23]–[25].

La interfaz no determina la arquitectura. Una caja de chat puede ser front-end de una búsqueda simple o de un sistema con RAG, tools y approvals. Deben documentarse capabilities reales, no el nombre comercial.

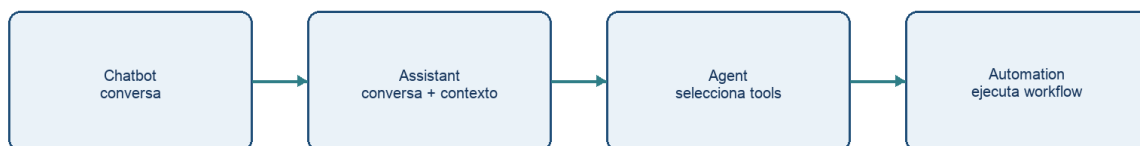
Cuándo conviene y cuándo no. Conviene un assistant para consulta y borradores; un agent cuando la selección dinámica aporta valor y existen límites, monitoring y approvals. No conviene autonomía amplia para acciones irreversibles o datos sensibles.

Si el proceso es estable, un workflow explícito suele ser más predecible y auditable.

Ejemplo correcto. Un assistant consulta políticas y redacta una respuesta; un humano aprueba. Un workflow registra la aprobación. El modelo no decide permisos.

Contraejemplo o error frecuente. Llamar agent a cualquier chatbot o asumir que “agentic” significa que puede actuar sin responsabilidad humana.

Qué puede simplificar el docente. Puede usarse responde → recomienda → selecciona herramientas → ejecuta flujo. Debe aclararse que no es una taxonomía universal.



No es una escala universal: autonomía, estado, herramientas y permisos importan.

Figura 7. Chatbot, assistant, agent y automation. Elaboración propia con base en las fuentes citadas en la sección. Texto alternativo: Cuatro bloques que comparan conversación, contexto, tool selection y ejecución de workflow.

Arquitectura básica de assistants

Explicación intuitiva. Un assistant útil es un sistema, no un prompt largo. Suele tener interfaz, identidad, instrucciones, modelo, context assembly, knowledge sources, tools, memoria, guardrails, logs y supervisión.

La analogía es una oficina: el modelo redacta y relaciona; la biblioteca aporta documentos; las tools ejecutan; los permisos determinan qué puede abrir; el registro permite reconstruir lo ocurrido.

Cada componente resuelve un problema diferente. Pedir al modelo que “no revele información” no reemplaza authorization. Pedir que cite no garantiza que la fuente recuperada sea vigente.

Explicación técnica. La orchestration recibe el request, identifica usuario y tenant, aplica authorization, recupera contexto, construye mensajes, llama al modelo, valida tool calls, ejecuta actions y registra resultados. Authentication responde quién es; authorization, qué puede ver o hacer.

Knowledge puede provenir de RAG, una API o una base estructurada. Memory puede ser conversation history, resumen, perfil autorizado o estado externo. Tool definitions actúan como contratos de entrada/salida; deben validarse con schemas y allowlists.

Guardrails se distribuyen en capas: filtros de entrada/salida, policy engine, permissions, human approval, rate limits y monitoring. Ninguna capa aislada ofrece seguridad completa.

Cuándo conviene y cuándo no. Conviene modularizar para probar retrieval, generation y actions por separado. No conviene incrustar secretos o permisos en system instructions ni exponer todas las tools a cada solicitud.

La arquitectura debe ser proporcional: una consulta pública puede ser sencilla; un assistant interno con datos sensibles requiere identidad, permisos y trazabilidad.

Ejemplo correcto. Aplicar permisos antes de recuperar documentos, pasar solo fragmentos autorizados y exigir aprobación antes de enviar una comunicación externa.

Contraejemplo o error frecuente. Poner todos los documentos y credenciales en el contexto y confiar en “no los muestres”.

Qué puede simplificar el docente. Puede mostrarse interfaz–orquestración–modelo–knowledge–tools–validación. Deben conservarse authentication/authorization y controles fuera del modelo.

Seguridad y protección de información

Explicación intuitiva. Proteger información implica clasificación, minimización, base legal, permisos, retención, cifrado, contratos, monitoreo y respuesta a incidentes. “No entrenar con tus datos” es solo una dimensión y debe verificarse según producto, plan y configuración actual [18].

Explicación técnica. Técnicamente, amenazas incluyen inyección de prompt, divulgación de información, acciones excesivas, abuso de herramientas y cadena de suministro [19], [30]. Las instrucciones no son un límite de seguridad robusto.

Cuándo conviene y cuándo no. Conviene usar cuentas empresariales aprobadas y datos mínimos. No conviene cargar datos personales, confidenciales o secretos en herramientas públicas sin autorización y controles.

Ejemplo, contraejemplo y simplificación docente. Correcto: sustituir identificadores y usar repositorio con permisos. Error: pegar base de clientes completa para “ahorrar tiempo”. Simplificación docente: si no debería estar en un correo público, no asumir que puede ir a cualquier AI.

Parte 2

Instrucciones y proyectos de asistentes

Explicación intuitiva. Las instrucciones definen misión, usuarios, alcance, fuentes, tono, reglas de rechazo, escalamiento y formato. Un proyecto de asistente añade activos y gobernanza: propietario, conjunto de pruebas, versiones, métricas y plan de mantenimiento.

Explicación técnica. Técnicamente, las instrucciones deben evitar contradicciones y secretos; las políticas críticas se implementan también fuera del modelo. Cada cambio requiere pruebas de regresión.

Cuándo conviene y cuándo no. Conviene separar reglas estables de información cambiante. No conviene escribir un prompt monolítico que intente resolver seguridad, datos y lógica de negocio.

Ejemplo, contraejemplo y simplificación docente. Correcto: “responde solo sobre políticas vigentes; cita; escala excepciones”. Error: “sé siempre correcto”. Simplificación docente: manual de funciones más fuentes y límites.

Concepto de apoyo: Retrieval-Augmented Generation, embeddings y knowledge bases

Pertinencia como concepto de apoyo. Es pertinente porque explica cómo un assistant consulta información actualizable y autorizada sin reentrenar el LLM.

Explicación intuitiva. Retrieval-Augmented Generation (RAG) separa dos trabajos: primero buscar evidencia; luego redactar con esa evidencia. La analogía es un bibliotecario que trae páginas y un redactor que las usa. Si el bibliotecario trae la página equivocada o el redactor la interpreta mal, la respuesta falla. RAG reduce dependencia del conocimiento paramétrico, pero no elimina confabulation [20].

Una knowledge base no es automáticamente todo lo que el modelo “sabe”. Es una colección administrada de fuentes con propietario, fecha, permisos y proceso de actualización.

Explicación técnica. El pipeline típico incluye ingestión, parsing, chunking, metadata, embeddings, indexación, query transformation, retrieval, filtering/reranking, context construction, generation y citation. Cada etapa necesita evals distintos. Retrieval recall mide si aparece la evidencia necesaria; generation faithfulness mide si el output se mantiene en ella.

Semantic search usa embeddings para recuperar fragmentos cercanos en el espacio vectorial; keyword search captura coincidencias exactas; hybrid search combina señales. Metadata filters aplican fecha, área y permisos. Chunk size y overlap afectan contexto y precisión.

RAG no es apropiado para toda consulta. Si se necesita el saldo exacto de una cuenta, una API o database query estructurada es preferible a buscar texto. Si la fuente contiene tablas complejas, puede requerirse parsing especializado.

Prompt injection indirecto puede entrar por documentos recuperados. El pipeline debe marcar content como no confiable, aplicar permissions antes de retrieval y separar instrucciones de evidencia [19], [30].

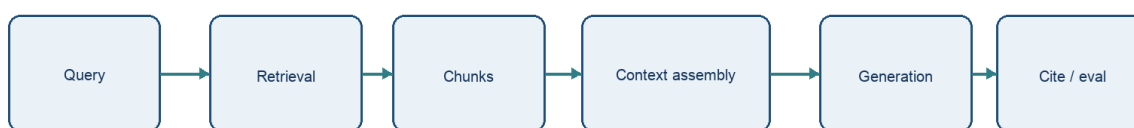
Cuándo conviene y cuándo no. Conviene para políticas, manuales, investigación y documentación cambiante con citas. No conviene cuando los datos son transaccionales, la fuente carece de gobernanza o una búsqueda exacta resuelve mejor.

La ausencia de evidencia debe producir abstención o escalamiento, no una respuesta inventada.

Ejemplo correcto. Recuperar la política vigente por país y fecha, devolver cita y enlace, y declarar que no hay respuesta si ningún fragmento supera el umbral.

Contraejemplo o error frecuente. Cargar archivos sin fecha ni permisos, llamar al conjunto knowledge base y afirmar que RAG garantiza cero hallucinations.

Qué puede simplificar el docente. Puede usarse bibliotecario + redactor. Deben conservarse las etapas, puntos de fallo, diferencia entre retrieval y generation y alternativa de API para datos exactos.



Cada etapa puede fallar; recuperar evidencia no garantiza una interpretación correcta.

Figura 8. Pipeline conceptual de Retrieval-Augmented Generation. Elaboración propia con base en las fuentes citadas en la sección. Texto alternativo: Query, retrieval, chunks, context assembly, generation y evaluación.

Integración con GPT, Gemini y Copilot

Explicación intuitiva. Las plataformas permiten configurar asistentes o agentes con instrucciones, archivos, conocimiento y herramientas, pero nombres, límites, licencias y disponibilidad cambian. Las capacidades deben describirse con fecha y documentación oficial, no como propiedades permanentes [22]–[25].

Explicación técnica. Técnicamente, la integración puede ser mediante interfaz sin código, conectores, funciones o API. Debe evaluarse identidad, permisos, residencia, retención, observabilidad, exportación y bloqueo de proveedor.

Cuándo conviene y cuándo no. Conviene escoger por ecosistema, gobernanza, costo total y necesidad, no por una respuesta anecdótica. No conviene comparar planes o límites sin verificar la versión vigente.

Ejemplo, contraejemplo y simplificación docente. Correcto: matriz de requisitos y piloto equivalente. Error: declarar un ganador universal. Simplificación docente: todos combinan modelo, contexto y herramientas; el detalle de producto es temporal.

Casos de uso pertinentes

- Asistente de consulta sobre políticas vigentes.

- Asistente para preparar respuestas que requieren aprobación.
- Asistente de investigación con base documental y citas.

Limitaciones, riesgos, controversias y errores frecuentes

- Inyección directa o indirecta desde documentos.
- Permisos excesivos y acciones irreversibles.
- Conocimiento desactualizado o mal fragmentado.
- Dependencia de características en vista previa.

Buenas prácticas

- Mínimo privilegio y aprobación para acciones sensibles.
- Fuentes con propietario, vigencia y permisos.
- Pruebas de recuperación separadas de pruebas de generación.
- Registro, monitoreo, canal de escalamiento y desactivación.

Demostración sugerida para el docente

Mostrar un asistente de consulta sobre tres documentos públicos: una respuesta sin recuperación, otra con RAG y cita, y un caso sin evidencia que debe escalar. Mostrar el diagrama de seguridad sin conectar datos reales.

Preguntas de reflexión o comprobación

- ¿Dónde se aplica realmente la autorización?
- ¿Qué diferencia recuperar de generar?
- ¿Cuándo una API es mejor que RAG?
- ¿Qué nivel de autonomía exige aprobación?

Ideas clave para convertir en diapositivas

- Chatbot–asistente–agente–automatización.
- Arquitectura por componentes.
- RAG en cinco pasos y puntos de fallo.
- Prompt no es perímetro de seguridad.
- Producto actual vs. fundamento estable.

Fuentes utilizadas en esta clase

Referencias principales: [18], [19], [20], [21], [22], [23], [24], [25], [30]. Las características de productos citadas deben verificarse de nuevo antes de impartir la sesión, porque pueden cambiar con versiones, planes y región.

Bibliografía

La bibliografía completa se encuentra en el siguiente archivo:

<https://evinracher.github.io/curso-sinergia/recursos/fundamentos-teoricos.pdf>